Gemini LinkedIn Scout

AI-Powered Networking Tool — Project Documentation

gemini-linkedin-scout.vercel.app • github.com/rishav-dutta

Solo Build	**PM · Designer · Developer**	**0 → 1 Product**	**TypeScript + React**

1. Overview

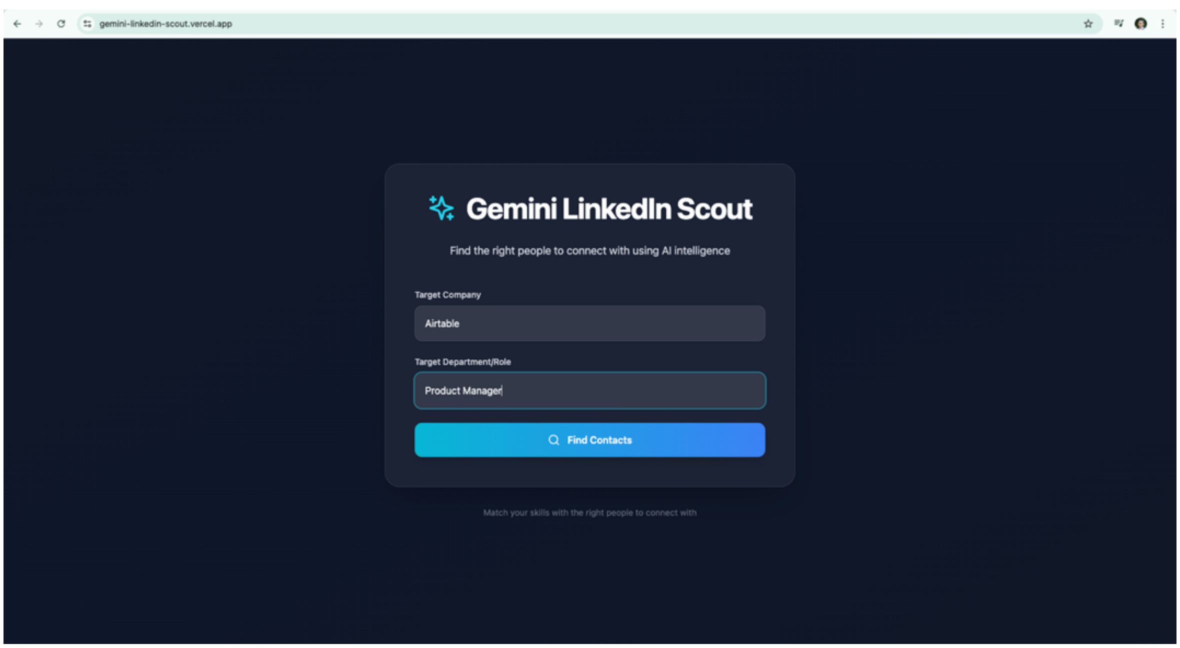
Gemini LinkedIn Scout is an AI-powered networking tool built to eliminate the manual effort of finding the right people to connect with at a target company. Given a company name and a target role or department, the tool automatically discovers relevant LinkedIn profiles, scores them against the user's resume, and surfaces the highest-relevance contacts — ranked with percentage match scores and AI-generated reasoning.

> **Core Problem Solved:** Most job seekers waste time manually searching LinkedIn and guessing who to reach out to. LinkedIn Scout automates discovery, relevance scoring, and prioritization in one end-to-end flow.

2. Product Flow

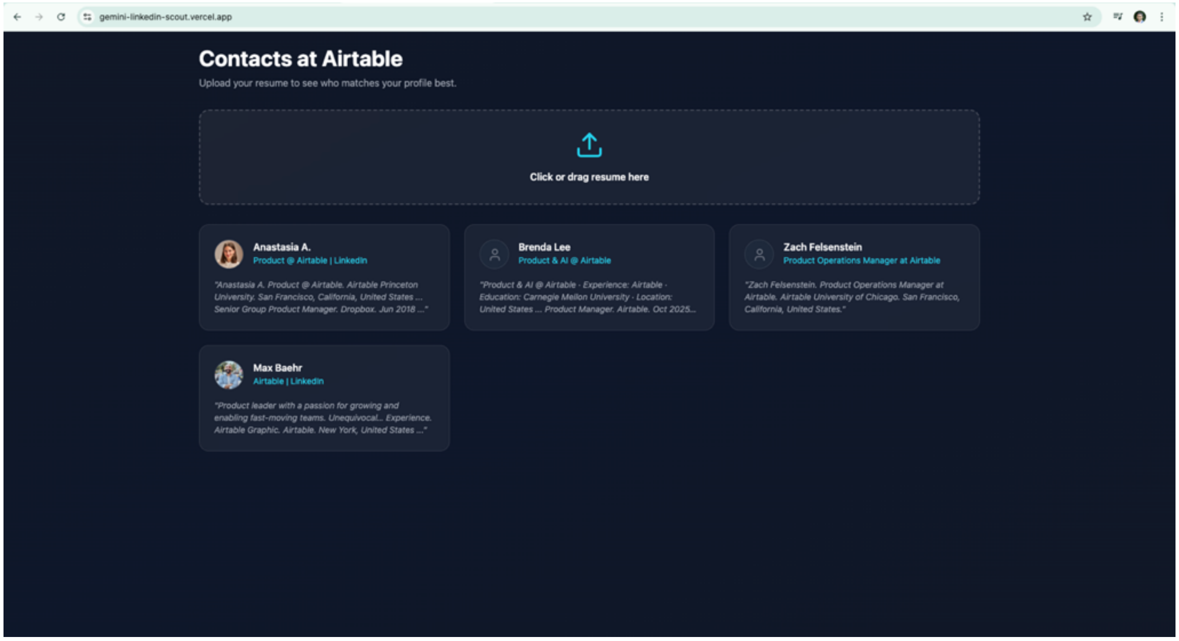
Step 1 — Targeting

The user enters a Target Company (e.g., "Airtable") and a Target Department/Role (e.g., "Product Manager") and clicks "Find Contacts." The backend kicks off the profile discovery n8n workflow via a webhook call.



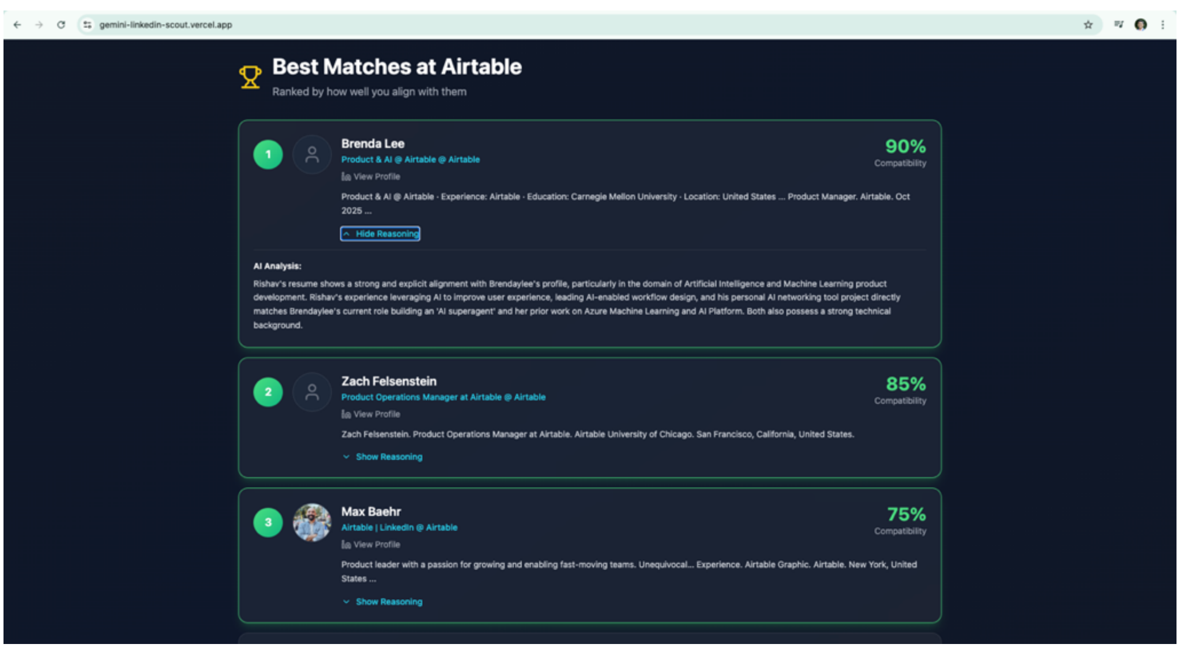
Step 2 — Resume Upload

The app returns a grid of discovered LinkedIn profiles at that company. The user uploads their resume (PDF) via drag-and-drop to trigger the Gemini-powered relevance scoring engine.



Step 3 — Ranked Results

The app displays a ranked "Best Matches" list ordered by compatibility score (e.g., 90%, 85%, 75%), with each contact's title, LinkedIn link, profile snippet, and an AI Analysis block explaining why that person is a strong match for the user's background. Framer Motion animations provide smooth transitions between states.



3. Tech Stack

Layer	Technology / Details
Frontend Language	TypeScript 4+ (93.5% of codebase)
Frontend Framework	React 18.3
Build Tool	Vite 5.4
Styling	Tailwind CSS 3.4
Animations	Framer Motion 12
Icons	lucide-react 0.344
Database Client	@supabase/supabase-js 2.57 (direct frontend integration)
Backend Orchestration	n8n (self-hosted on DigitalOcean)
AI / LLM	Google Gemini (via n8n AI Agent node)
Data Scraping	Apify — harvestapi/linkedin-profile-scraper actor
Database	Supabase (PostgreSQL)
Server	DigitalOcean Droplet (Ubuntu 22.04, NYC1, 1 GB RAM)
Deployment	Vercel (auto-deploy from main, instant rollback)
Analytics	Vercel Web Analytics
Dev Tools	Claude, Cursor, ESLint, TypeScript compiler

4. Architecture

The application is a four-layer system: a TypeScript/React frontend on Vercel communicates exclusively through webhook HTTP calls to a self-hosted n8n instance on DigitalOcean. n8n orchestrates all backend logic including scraping, data storage, and AI scoring. Supabase provides the PostgreSQL data layer, and the Supabase JS client is also embedded directly in the frontend for read operations.

> **Stack in one line:** React (Vite + TS) → n8n webhooks → Apify scraper + Gemini AI → Supabase PostgreSQL

4.1 Frontend — React + TypeScript on Vercel

- TypeScript throughout (93.5% of codebase) with strict tsconfig settings
- Vite 5.4 for fast local dev and optimised production builds
- Tailwind CSS for utility-first styling; Framer Motion for page/card animations
- lucide-react for consistent iconography

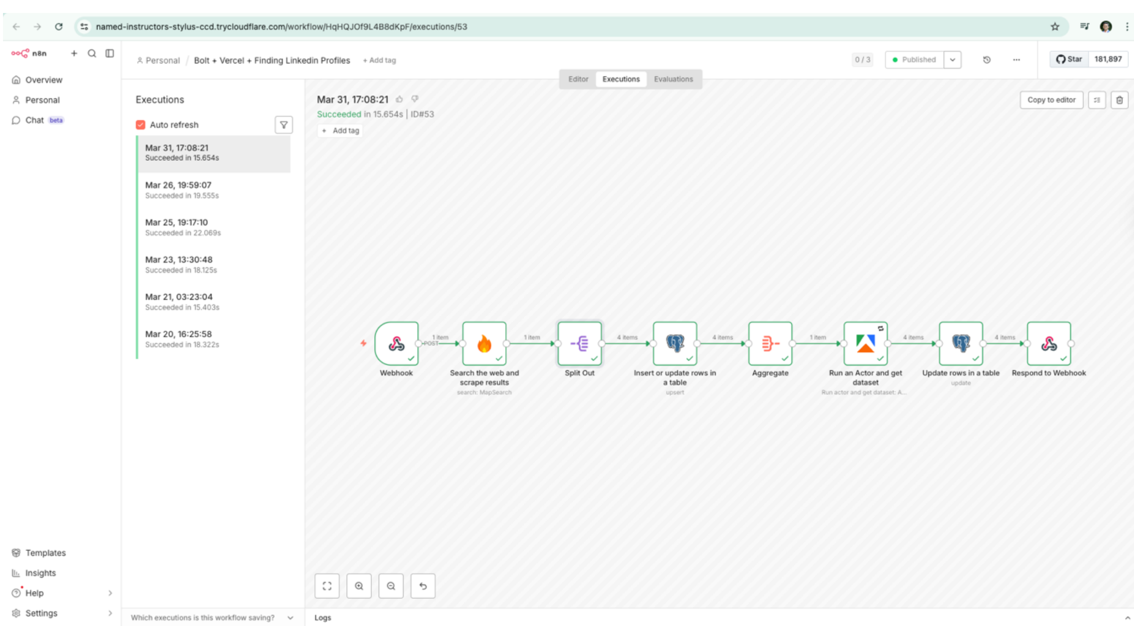
- @supabase/supabase-js client enables the frontend to read ranked results directly from Supabase after n8n writes them
- All backend communication via HTTPS webhook calls to n8n — no custom API server

4.2 Backend — n8n Workflows on DigitalOcean

Two production workflows run on a self-hosted DigitalOcean Droplet (1 GB RAM, NYC1, Ubuntu 22.04). Choosing self-hosted over n8n Cloud eliminated per-execution pricing limits and timeout constraints on long Apify scraping jobs.

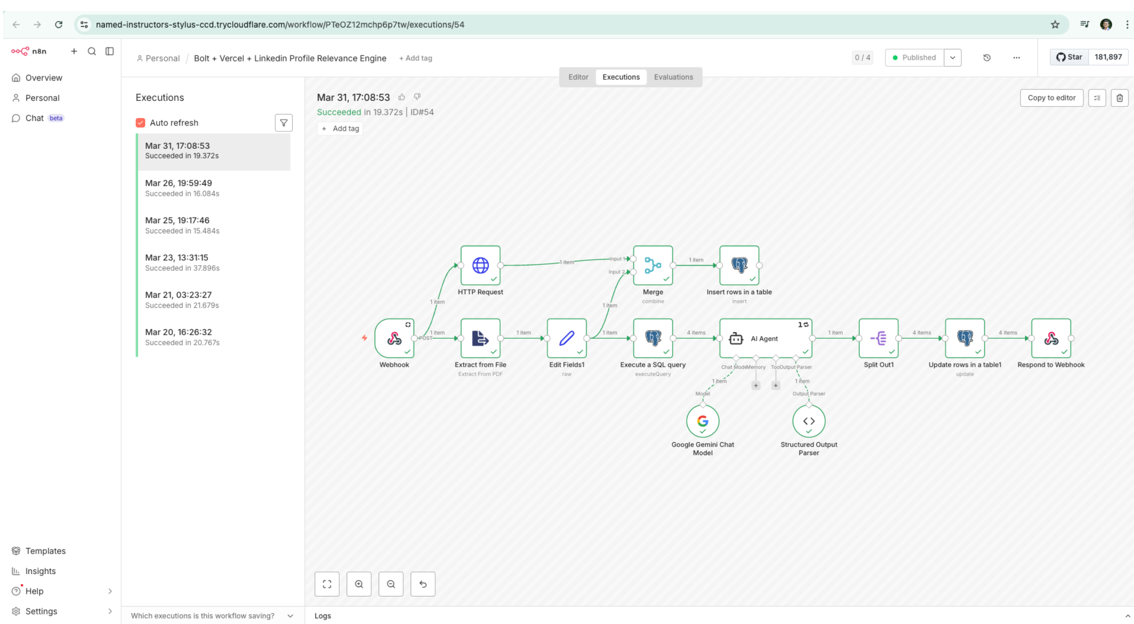
Workflow 1: LinkedIn Profile Finder

#	Node	Purpose
1	**Webhook**	Receives company + role payload from the React frontend
2	**Search the web & scrape results**	Searches for LinkedIn profiles matching the company/role query
3	**Split Chat**	Parses and splits scraped results into individual records
4	**Insert / Update rows in table**	Upserts raw profile data into Supabase linkedin_leads table
5	**Aggregate**	Consolidates all records before the enrichment step
6	**Run Apify Actor & get dataset**	Invokes Apify actor for enriched full-profile LinkedIn data
7	**Update rows in table**	Enriches Supabase rows with full_profile and profile_image data
8	**Respond to Webhook**	Returns the profile list to the React frontend



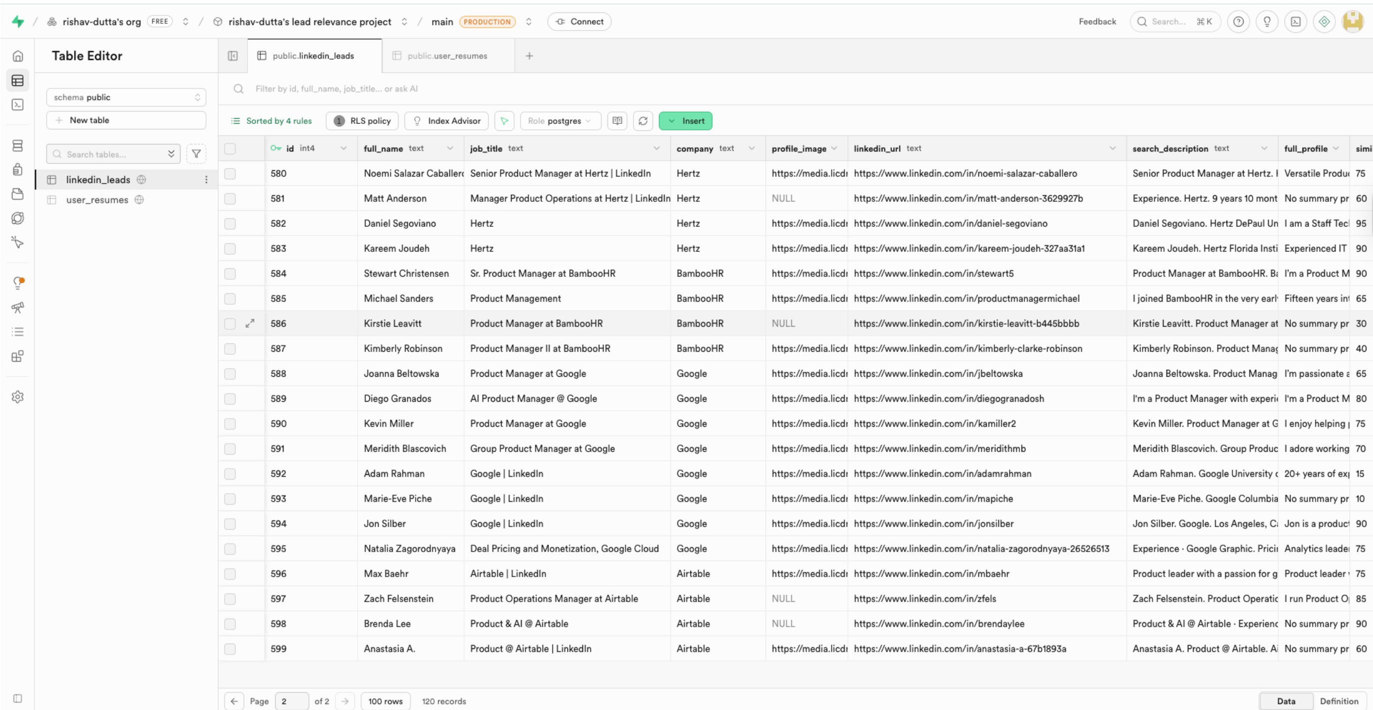
Workflow 2: LinkedIn Profile Relevance Engine

#	Node	Purpose
1	**Webhook**	Receives resume upload trigger from frontend
2	**HTTP Request**	Fetches stored profiles from Supabase via REST API
3	**Extract from File**	Parses the uploaded PDF resume into text
4	**Edit Fields**	Normalises and flattens resume fields for the AI prompt
5	**Execute SQL query**	Retrieves target company profiles from Supabase
6	**Merge**	Combines resume data with LinkedIn profile records
7	**Insert rows in table**	Stores the merged scoring input set
8	**AI Agent (Gemini)**	Scores each profile vs the resume; generates match reasoning
9	**Structured Output Parser**	Extracts JSON {score, reasoning} from the Gemini response
10	**Split Out**	Separates individual scored profile results
11	**Update rows in table**	Writes similarity scores back to linkedin_leads in Supabase
12	**Respond to Webhook**	Returns ranked results to the React frontend



4.3 Data Layer — Supabase (PostgreSQL)

Table	Schema / Purpose
linkedin_leads	full_name, job_title, company, linkedin_url, profile_image, search_description, full_profile, similarity_score (INT 0–100), scoring_reasoning, search_id
user_resumes	Stores uploaded resume files (PDF) tied to each search session for the scoring pipeline



id int4	full_name text	job_title text	company text	profile_image	linkedin_url text	search_description text	full_profile	simi
580	Noemi Salazar Caballer	Senior Product Manager at Hertz | LinkedIn	Hertz	https://media.licd	https://www.linkedin.com/in/noemi-salazar-caballero	Senior Product Manager at Hertz. I	Versatile Produ	75
581	Matt Anderson	Manager Product Operations at Hertz | LinkedIn	Hertz	NULL	https://www.linkedin.com/in/matt-anderson-3629927b	Experience. Hertz. 9 years 10 mont	No summary pr	60
582	Daniel Segoviano	Hertz	Hertz	https://media.licd	https://www.linkedin.com/in/daniel-segoviano	Daniel Segoviano. Hertz DePaul Un	I am a Staff Tec	95
583	Kareem Joudeh	Hertz	Hertz	https://media.licd	https://www.linkedin.com/in/kareem-joudeh-327aa31a1	Kareem Joudeh. Hertz Florida Insti	Experienced IT	90
584	Stewart Christensen	Sr. Product Manager at BambooHR	BambooHR	https://media.licd	https://www.linkedin.com/in/stewart5	Product Manager at BambooHR. B	I'm a Product M	90
585	Michael Sanders	Product Management	BambooHR	https://media.licd	https://www.linkedin.com/in/productmanagermichael	I joined BambooHR in the very earl	Fifteen years in	65
586	Kirstie Leavitt	Product Manager at BambooHR	BambooHR	NULL	https://www.linkedin.com/in/kirstie-leavitt-b445bbbb	Kirstie Leavitt. Product Manager at	No summary pr	30
587	Kimberly Robinson	Product Manager II at BambooHR	BambooHR	https://media.licd	https://www.linkedin.com/in/kimberly-clarke-robinson	Kimberly Robinson. Product Mana	No summary pr	40
588	Joanna Beltowska	Product Manager at Google	Google	https://media.licd	https://www.linkedin.com/in/jbeltowska	Joanna Beltowska. Product Manag	I'm passionate	65
589	Diego Granados	AI Product Manager @ Google	Google	https://media.licd	https://www.linkedin.com/in/diegogranadosh	I'm a Product Manager with experi	I'm a Product M	80
590	Kevin Miller	Product Manager at Google	Google	https://media.licd	https://www.linkedin.com/in/kamiller2	Kevin Miller. Product Manager at G	I enjoy helping	75
591	Meridith Blascovich	Group Product Manager at Google	Google	https://media.licd	https://www.linkedin.com/in/meridithmb	Meridith Blascovich. Group Produc	I adore working	70
592	Adam Rahman	Google | LinkedIn	Google	https://media.licd	https://www.linkedin.com/in/adamrahman	Adam Rahman. Google University	20+ years of ex	15
593	Marie-Eve Piche	Google | LinkedIn	Google	https://media.licd	https://www.linkedin.com/in/mapiche	Marie-Eve Piche. Google Columbia	No summary pr	10
594	Jon Silber	Google | LinkedIn	Google	https://media.licd	https://www.linkedin.com/in/jonsilber	Jon Silber. Google. Los Angeles, C	Jon is a produc	90
595	Natalia Zagorodnyaya	Deal Pricing and Monetization, Google Cloud	Google	https://media.licd	https://www.linkedin.com/in/natalia-zagorodnyaya-26526513	Experience · Google Graphic. Prici	Analytics leade	75
596	Max Baehr	Airtable | LinkedIn	Airtable	https://media.licd	https://www.linkedin.com/in/mbaehr	Product leader with a passion for g	Product leader	75
597	Zach Felsenstein	Product Operations Manager at Airtable	Airtable	NULL	https://www.linkedin.com/in/zfels	Zach Felsenstein. Product Operati	I run Product O	85
598	Brenda Lee	Product & AI @ Airtable	Airtable	NULL	https://www.linkedin.com/in/brendaylee	Product & AI @ Airtable · Experienc	No summary pr	90
599	Anastasia A.	Product @ Airtable | LinkedIn	Airtable	https://media.licd	https://www.linkedin.com/in/anastasia-a-67b1893a	Anastasia A. Product @ Airtable. A	No summary pr	60

4.4 AI Layer — Google Gemini

- Ingests the user's parsed resume alongside each candidate LinkedIn profile
- Reasons about alignment across domain expertise, skills, role history, and stated interests
- Outputs a structured compatibility score (integer 0–100) + plain-language AI Analysis paragraph per contact
- n8n's Structured Output Parser enforces JSON schema compliance, eliminating parsing failures from LLM preamble

5. Key Product Decisions

Why n8n over a custom API server?

n8n allowed rapid workflow prototyping without writing and maintaining a full backend codebase. Each node is visual and independently debuggable, which dramatically cut development time for a solo build. The execution logs also made debugging the Gemini scoring pipeline significantly faster.

Why self-hosted n8n on DigitalOcean vs. n8n Cloud?

Full control over the execution environment, no per-execution pricing limits, and the ability to run long-running Apify scraping jobs without hitting cloud timeout constraints.

Why Supabase?

Instant PostgreSQL with a UI-first table editor, a REST API generated automatically from the schema, and a JS client that works directly from the React frontend — no custom ORM or migration pipeline needed for a solo build. The supabase/migrations folder in the repo tracks schema changes.

Why score contacts rather than just surface them?

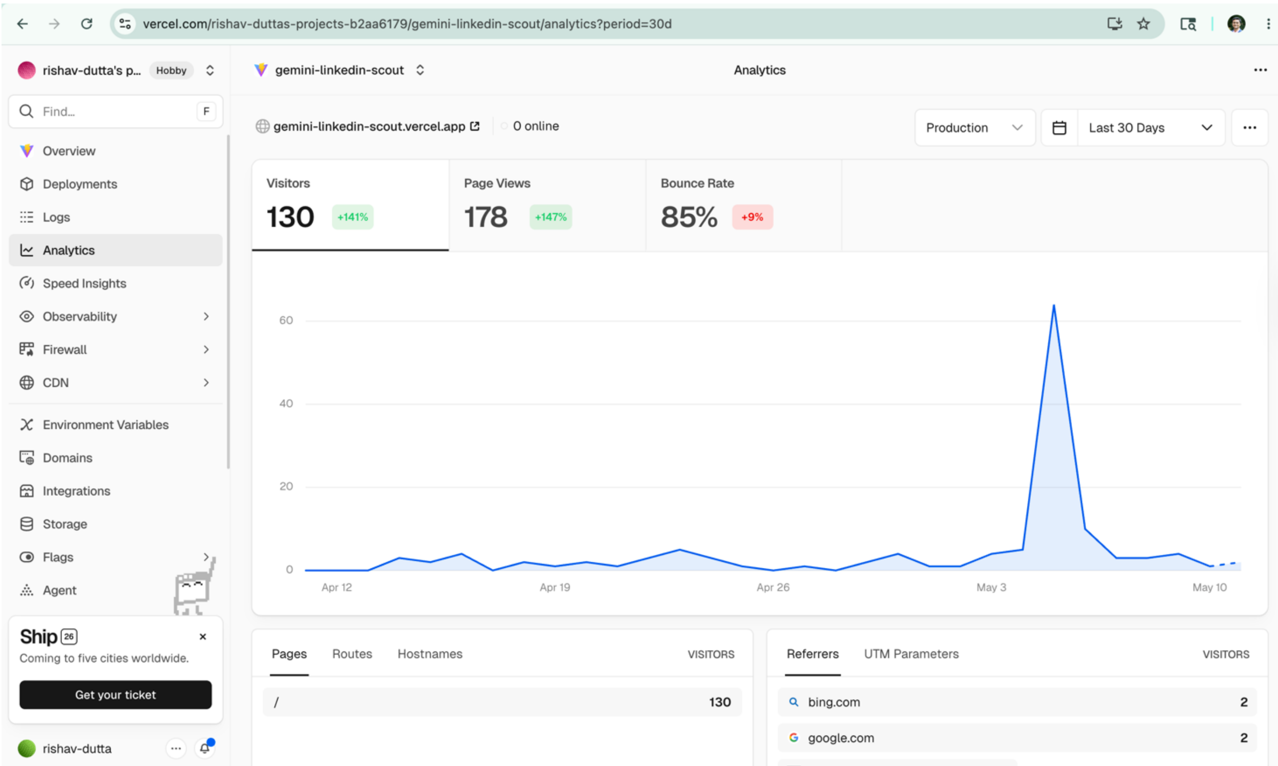
Raw scraping results are noisy. Ranking by resume-to-profile alignment makes the outreach list immediately actionable — the user knows exactly who to prioritise and why, mirroring how a recruiter or hiring manager evaluates fit. This is also the core differentiator of the product.

Why TypeScript over plain JavaScript?

Type safety on the Supabase query responses and webhook payloads caught several schema mismatches during development. With 93.5% TypeScript coverage across 75 commits, the codebase is significantly more maintainable for future feature additions.

6. Metrics & Deployment (Last 30 Days)

Metric	Value
Status	Live in production
Visitors (30d)	130 (+141% period-over-period)
Page Views (30d)	177 (+146% period-over-period)
Bounce Rate	85% (+9%)
Top Referrers	bing.com (2), google.com (2)
Commits	75 commits on main
Open Pull Requests	1
Error Rate	0%
Deployment model	Auto-deploy from main via Vercel Git integration
Rollback	Instant Rollback enabled on Vercel
Server uptime	DigitalOcean Droplet — n8n-backend-production, NYC1



7. Challenges & Learnings

LinkedIn scraping reliability

Apify actor results vary in completeness depending on profile privacy settings. The workflow handles missing profile_image and full_profile fields gracefully with NULL defaults and falls back to search_description as the scoring input when full_profile is unavailable.

LLM structured output consistency

Gemini responses occasionally included conversational preamble before the JSON payload. Adding n8n's Structured Output Parser node enforced schema compliance and eliminated downstream parsing failures entirely. The fix also improved scoring latency by removing retry logic that had been needed for malformed responses.

Frontend–backend latency

The full pipeline (scrape → enrich → score) takes 15–30 seconds. Framer Motion loading states on both the profile discovery and relevance scoring phases prevent the experience from feeling broken during processing.

Solo full-stack scope management

Acting as PM, designer, and developer simultaneously required strict prioritization. The MVP deliberately omitted user authentication and saved-search history to ship a working end-to-end flow first. The 75-commit history reflects iterative, production-focused development rather than big-bang releases.

8. Potential Next Features

- Saved search history per user (requires auth layer — Supabase Auth is already available in the stack)
- Draft InMail / connection request generator based on the matched contact's profile + user's resume
- Batch company search — run the discovery pipeline for multiple companies simultaneously
- Score threshold filter — show only contacts above a minimum compatibility percentage
- Export ranked contact list to CSV for CRM or outreach tool import
- Webhook-based refresh to re-score contacts when the user uploads an updated resume